Reinforcement Learning for Stochastic Control and Games in Algorithmic Trading

by

Xin Ning

PREVIEW

A thesis submitted in conformity with the requirements
for the degree of Doctor of Philosophy

Department of Statistical Science
University of Toronto

© Copyright 2023 by Xin Ning

Reinforcement Learning for Stochastic Control and Games in Algorithmic Trading

Xin Ning
Doctor of Philosophy

Department of Statistical Science
University of Toronto
2023

Abstract

Algorithmic trading in electronic markets is a well-studied field in finance with a plethora of different possible approaches. This thesis explores how agents participating in electronic markets should optimally trade when accounting for latent factors and the behaviour of other participating agents. It investigates the problem by developing a modified Deep Q-Learning method for a single agent trader, generalizing the approach for multi-agent games using Nash equilibria, and constructing a new method for generating arbitrage-free implied volatility surfaces used for derivative pricing.

The thesis contains three main parts. In the first part we take a model free approach to optimal trade execution and develop a variation of Deep Q-Learning to estimate the optimal actions of a trader. The model is a fully connected Neural Network trained using Experience Replay and Double DQN with input features given by the current state of the limit order book, other trading signals, and available execution actions, while the output is the Q-value function estimating the future rewards under an arbitrary action. We apply our model to nine different stocks and find that it significantly outperforms a baseline approach.

In the second part, we generalize this approach into the multi-agent setting by developing a new data efficient Deep Q-learning methodology for model-free learning of Nash equilibria for general-sum stochastic games. The algorithm uses a local linear-quadratic approximation of the stochastic game, which leads to analytically solvable optimal actions. The approximation is parameterized by deep neural networks to provide sufficient flexibility to learn the environment without the need to experience all state-action pairs. We study symmetry properties of the algorithm stemming from label-invariant stochastic games and as a proof of concept, apply our algorithm to learning optimal trading strategies in various simulated competitive electronic markets.

In the third part, we take the first steps in extending our approach into the derivatives market by proposing a new hybrid method for generating arbitrage-free implied volatility (IV) surfaces, which is a key feature used in derivative pricing. Our approach combines model-free Variational Autoencoders (VAEs) with continuous time stochastic differential equation (SDE) driven models. We focus on two classes of SDE models: regime switching models and Lévy additive processes. By

PREVIEW

projecting historical surfaces onto the space of SDE model parameters, we obtain a distribution on the parameter subspace faithful to the data on which we then train a VAE. Arbitrage-free IV surfaces are then generated by sampling from the posterior distribution on the latent space, decoding to obtain SDE model parameters, and finally mapping those parameters to IV surfaces. We further refine the VAE model by including conditional features and demonstrate its superior generative out-of-sample performance.

PREVIEW

Acknowledgements

Coming into a a PhD program directly from a undergraduate program is never easy, and I found myself struggling quickly in the early days of the program. I felt directionless, and completely overwhelmed by the new atmosphere. However, I was only able to overcome the hurdles and arrive at where I am at now with the help of my mentors, colleagues, and friends.

First and foremost, I wish to thank my PhD supervisor Professor Sebastian Jaimungal. Never a day goes by where I'm not reminded of how extremely fortuitous it was for me to reach out and take on that very first research project. His guidance and mentorship have been irreplaceable for me during my entire time at the University of Toronto. Sebastian's unending dedication to his craft have inspired me on countless occasions and it is no exaggeration to say that every part of this thesis have been influenced by him in some way.

I would also like to thanks the members of my committee Professor Yuchong Zhang, Professor David Duvenaud, and Professor Chi-Guhn Lee for their insightful comments and feedback which were able to make the thesis significantly stronger and more coherent.

This thesis could also have not been possible with the contributions of all my collaborators and fellow University of Toronto alumni: Franco Ho Ting Lin, Philippe Casgrain, and Xiaorong Zhang, as well as Maxime Bergeron from Riskfuel Analytics. Their work have proved to be invaluable to this thesis.

I also wish to thank all my friends and colleagues both in and outside of the Mathematical Finance group, including: Arvind, Ali, Michaël, Bill, Yanbo, and many others. Your help and patience have been a great support for me over the years.

Thank you to all the other staff and faculty at the Statistics Department at the University of Toronto for creating a wonderful program and a great place to learn.

Finally I wish to thank my brother, mother, and father for putting up with me during this time. Your love and support are the pillars that support what I am today.

PREVIEW

Contents

1 Introduction **1**

1.1 The Limit Order Book . . . 1

1.2 Algorithmic Trading . . . 2

1.3 Reinforcement Learning . . . 3

1.3.1 Q-Learning . . . 4

1.3.2 Dyna-Q Learning . . . 5

1.4 Neural Networks . . . 7

1.5 Literature Review . . . 8

1.6 Outline and Contributions . . . 9

1.7 Remarks . . . 10

2 Double Deep Q-Learning for Optimal Execution **11**

2.1 Introduction . . . 11

2.2 Background . . . 12

2.2.1 Optimal Execution . . . 12

2.2.2 Classical Almgren-Chriss . . . 12

2.2.3 Reinforcement Learning . . . 16

2.2.4 Deep Q-Learning . . . 16

2.3 Optimal Execution in a Reinforcement Learning Setting . . . 17

2.3.1 States . . . 17

2.3.2 Actions . . . 17

2.3.3 Rewards . . . 17

2.4 Network Architecture . . . 18

2.4.1 Basic Network Architecture . . . 18

2.4.2 Experience Replay . . . 20

2.4.3 Main and Target Networks . . . 20

2.4.4 Double DQN . . . 20

2.4.5 Treatment of the Zero Ending Inventory Constraint . . . 21

2.4.6 Price Impact . . . 22

2.5 Training Method . . . 23

2.5.1 ϵ-Greedy . . . 24

2.5.2 Hyperparameters . . . 24

2.5.3 Pre-training . . . 24

PREVIEW

2.6 Features . . . 24
2.6.1 Transforming Inventory-Action Domain . . . 25
2.7 Results . . . 26
2.7.1 Data . . . 26
2.7.2 Evaluation Metrics . . . 26
2.7.3 Experiments . . . 27
2.7.4 Statistical results . . . 28
2.8 Conclusions . . . 34

3 Deep Q-Learning for Nash Equilibria: Nash-DQN 35
3.1 Introduction . . . 35
3.2 Nash Equilibrium . . . 36
3.3 Model Setup . . . 37
3.4 Optimality Conditions . . . 38
3.5 Policy Gradient Algorithms . . . 39
3.5.1 Simple Policy Gradient . . . 39
3.5.2 Actor-Critic Algorithm . . . 41
3.5.3 Normalized Advantage Functions . . . 42
3.6 Locally Linear-Quadratic Nash Q-Learning . . . 44
3.6.1 Simplifying Game Structures . . . 46
3.7 Implementation of Nash Actor-Critic Algorithm . . . 48
3.8 Experiments . . . 49
3.8.1 Features . . . 51
3.8.2 Network Details . . . 51
3.8.3 Baseline - Fictitious Play . . . 51
3.8.4 Optimization Improvements . . . 52
3.8.5 Results . . . 53
3.9 Sensitivity Analysis . . . 54
3.9.1 Baseline . . . 55
3.9.2 High Transaction Costs . . . 55
3.9.3 No Price Impact . . . 57
3.9.4 Slower Temporary Impact Decay . . . 58
3.9.5 Running Inventory Penalty - High . . . 59
3.9.6 Running Inventory Penalty - Low . . . 60
3.9.7 High Volatility . . . 61
3.9.8 Two-agent Case . . . 62
3.9.9 Additional Remarks . . . 63
3.10 Conclusions . . . 63

4 Arbitrage-Free Implied Volatility Surface Generation with Variational Autoencoders 64
4.1 Introduction . . . 64
4.2 Implied Volatility Surfaces . . . 66
4.3 Model Setup and Estimation Procedure . . . 68

PREVIEW

4.3.1 Wasserstein Distance . . . 69
4.3.2 Candidate Risk Neutral Density . . . 70
4.3.3 Effect of α . . . 71
4.4 Class of Stochastic Drivers . . . 71
4.4.1 CTMC . . . 72
4.4.2 Lévy Additive Processes . . . 73
4.5 Model Parameter Generation . . . 75
4.5.1 Variational Autoencoder (VAE) . . . 75
4.5.2 β-VAE . . . 76
4.5.3 Latent Sampling . . . 77
4.5.4 Conditional Variational Autoencoder (CVAE) . . . 78
4.6 Results . . . 79
4.6.1 Market Data . . . 79
4.6.2 Market Data Conversion . . . 79
4.6.3 SDE Model Specifics . . . 81
4.6.4 Structure of the A matrix . . . 81
4.6.5 Goodness of Fit . . . 82
4.6.6 VAE Model Specifics . . . 84
4.6.7 Benchmarks . . . 84
4.6.8 Evaluation Metric . . . 84
4.6.9 Sinkhorn Algorithm . . . 85
4.6.10 Results Summary . . . 87
4.6.11 VAE's Latent Space . . . 89
4.6.12 CVAE Results . . . 91
4.7 Conclusions . . . 92

5 Conclusion and Future Work **98**
5.1 Summary of Contributions . . . 98
5.2 Future Works . . . 100

Bibliography **101**

PREVIEW

List of Tables

2.1 Relative P&L performance for all stocks with respect to TWAP strategy. 31

3.1 Pay-off Table of NEP with three Nash Equilibria (bold). Pay-offs are denoted as "Agent 1/Agent 2". 37
3.2 Asset price process, price impact, and risk preference parameters. 53
3.3 Asset price process, price impact, and risk preference parameters for the baseline simulation. 55
3.4 Asset price process, price impact, and risk preference parameters for a simulation with significantly higher transaction and liquidation costs. Changes from the baseline are highlighted in red. 56
3.5 Asset price process, price impact, and risk preference parameters for a simulation with no permanent nor temporary price impact from trades from all agents. Changes from the baseline are highlighted in red. 57
3.6 Asset price process, price impact, and risk preference parameters for a simulation with slow temporary impact decay. Changes from the baseline are highlighted in red. 58
3.7 Asset price process, price impact, and risk preference parameters for a simulation with high running inventory penalty. Changes from the baseline are highlighted in red. 59
3.8 Asset price process, price impact, and risk preference parameters for a simulation with low running inventory penalty. Changes from the baseline are highlighted in red. 60
3.9 Asset price process, price impact, and risk preference parameters for a simulation with higher instantaneous volatility. Changes from the baseline are highlighted in red. 61
3.10 Asset price process, price impact, and risk preference parameters for a simulation with only two players. There are no otherwise changes simulation parameters. 62
3.11 Asset price process, price impact, and risk preference parameters for a simulation with no transaction costs. Changes from the baseline are highlighted in red. 63

4.1 Summary of characteristic functions within a given maturity period. 75
4.2 Median rmse ($\times 10^{-5}$) for the collection of models and FX pairs. 82
4.3 Wasserstein metrics ($\times 10^{-2}$) for varying levels of β, latent dimensionality, and currency pair. Three benchmarks are shown here for reference: the Wasserstein metric ($\times 10^{-2}$) between the test set and the VAE-IV, PCA, and training empirical models as described in Section 4.6.7. Bold numbers are the smallest metric within each subgrid. 93

PREVIEW

4.4 Average Wasserstein's metric ($\times 10^{-2}$) across all β values for different number of latent dimensions used for the CTMC-CVAE, CTMC-VAE, and benchmarks models IV-VAE, PCA, and Empirical for three currency pairs. 95
4.5 Wasserstein metrics ($\times 10^{-2}$) for varying levels of β, latent dimensionality, and currency pair using the CVAE with the CTMC model with VIX as predictor compared with the vanilla CTMC-VAE and benchmark approaches (B) IV-VAE and CTMC-PCA trained on the period from September 18th, 2012 to May 09, 2016. The Wasserstein metric ($\times 10^{-2}$) between the training and tests sets (Empirical) are presented here for reference. 96
4.6 Normalizations performed on model parameters before input into VAE. $\bar{\mu}(\cdot)$ and $s(\cdot)$ are the empirical mean and standard deviation respectively. 97

PREVIEW

List of Figures

1.1 The LOB of Facebook Inc. stock at 10:36 on 28 Mar, 2018 as a buy MO of size 3,000 arrives. The blue (red) bars represent available volume of sell (buy) orders at the corresponding price. The yellow bars in the middle panel indicate which LOs match the incoming MOs. 2

1.2 Graphical representation of the interactions between the environment and agent. . . 3

1.3 Matrix representation of the Q-Value function for a problem with five possible discretized actions and six states. 5

1.4 Graphical representation of a single neuron inside of neural network. The arbitrary non-linear function f is applied to a linear combination of upstream nodes plus a bias and outputs its value as a downstream node. 7

2.1 Execution procedure and full update procedure used when training the network. Note that when we are training our network, ϵ-Greedy is added to the Execution step. In the update procedure, we split the process into 3 steps, (1) Getting the Q-estimate given the next state $s^{s,x}$, (2) Computing the Loss after we have obtained the Q-estimate and (3) Computing the gradients and updating our network weights. 19

2.2 Inventory and Action Transformation. For explicit formula see Appendix 2.6.1. . . . 25

2.3 NTAP – Optimal Actions for All Possible States with Features: Time, Inventory, Price — Each plot from left to right denotes a change from low to high (normalized) price. Within each panel, the x and y axis denote time and inventory remaining, respectively. The color of individual squares denote the amount of shares sold, lighter being more shares, darker being less. 28

2.4 SMH – Optimal Actions for All Possible States with Features: Time, Inventory, Price — Each plot from left to right denotes a change from low to high price. The x and y axis denote the time and inventory remaining respectively. The color of individual squares denote the amount of shares sold, lighter being more shares, darker being less. 29

2.5 NTAP – Average Optimal Actions Split on QV and Price with Features: Time, Inventory, Price, QV. — For each individual plot, the x-axis denotes time intervals and y-axis inventory remaining. Moving across graphs, the x-axis denote changes from low to high QV, whereas the changes in the y-axis denote changes in price from low to high. 30

2.6 Distribution of the relative P&L (basis points) with various features (AAPL, AMZN, FB, GOOG, and INTC). 32

PREVIEW

2.7 Distribution of the relative P&L (basis points) with various features (MSFT, NTAP, SMH, and VOD). 33

3.1 Visual representation of the process used to train using an Actor-Critic Algorithm. The actor first determines the optimal action to maximize their rewards by maximizing their approximate Q-value function. The the environment determines rewards obtained by the agent and its next state. The critic then uses this information to make a better approximation of the Q-value function which is then optimized by the agent. 41

3.2 Average loss for different β values. Optimal value is $\beta = 100$. 52

3.3 Optimal trade execution heatmaps of the Nash-DQN and Fictitious Play policies as a function of time, inventory, price, and cumulative transient impact (Y_t), other features were assumed to be equal to zero. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. 53

3.4 Results of deviating from the Fictitious Play policy (i) and the Nash-DQN Policy (ii) when taking the total rewards obtained through randomly initialized simulated environments. Each data point represents the average of 1,000 simulations, repeated 100 times. 54

3.5 Optimal trade execution heatmaps of a Nash-DQN policy as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for the baseline simulation. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 54

3.6 Optimal trade execution heatmaps of a Nash-DQN policy under high transaction costs as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for the simulation with increased transaction costs. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 56

3.7 Optimal trade execution heatmaps of a Nash-DQN policy under no price impact as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for the simulation with no price impact from trades. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 57

PREVIEW

3.8 Optimal trade execution heatmaps of a Nash-DQN policy under a slow rate of temporary impact decay as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for the simulation with slow temporary impact decay. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 58
3.9 Optimal trade execution heatmaps of a Nash-DQN policy under high running inventory penalty as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for the simulation with some running inventory penalty. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 59
3.10 Optimal trade execution heatmaps of a Nash-DQN policy under a low but non-zero inventory penalty as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for the simulation with some running inventory penalty. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 60
3.11 Optimal trade execution heatmaps of a Nash-DQN policy with high intrinsic volatility as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for the simulation higher instantaneous volatility. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 61
3.12 Optimal trade execution heatmaps of a Nash-DQN policy with only two agents as a function of time, inventory, price, cumulative transient impact (Y_t), and other players' total order flow for a two-agent simulation. Within each panel, subplots corresponds to price levels $\$9.50, \$9.75, \ldots, \$10.50$ from left to right. The dotted line shows the threshold where the agent switches from buying to selling. The first row of heatmaps assumes $\mathcal{T} = 0$ and the second row of heatmaps assumes $Y_t = 0$. 62

4.1 Flow chart of algorithm starting from raw IV surface data to generated surfaces. . . 65
4.2 Example implied volatility surfaces for Apple Inc. (ticker = AAPL) for three trading days March 03, 2014 (left), March 10, 2014 (mid), and March 14, 2014 (right). Axis represents the strike price (in USD), time-to-maturity (in days), and implied volatility. 67
4.3 Fits of the CTMC model to the first maturity (1M) of a particular day (2012-05-16) at varying level of the regularizer α. The left panel showcases the fits to the IV surface and the right panel the corresponding risk-neutral density. 71

PREVIEW

4.4 Typical fits of the CTMC model to the IVs (top) and densities (bottom) of AUD-USD data with root mean squared error in the 50th (blue) and 90th (red) quantiles with larger quantiles indicating relatively "poorer" fits. Strike ranges differ on different days. 74
4.5 VAE architecture. 76
4.6 Scatter plots of samples from the approximate posterior $q_\phi(\mathbf{z}|\boldsymbol{X})$ from the first two latent dimensions of a VAE trained using different values of the hyperparameter β. . 77
4.7 Visual representation of the a Conditional Variational Autoencoder (CVAE). The diagram describing the process of encoding a single digit (7) into the CVAE's latent space by modulating the input parameters with the label then conditionally sampling a digit (7) by modulating the decoder with the conditional label we wish to sample from the latent space. 78
4.8 Implied volatility curves for the five worst fitted days in the data set based on the cumulative squared distance between the fitted model curve (blue line) and data (black dots) sorted from the least (top) to most (bottom). Shaded region represents the bid/ask ranges for call option prices at the corresponding strike-maturity pairs. Columns represents the surface slices of the IV surface at different time-to-maturities and rows the IV surface on different trading days. 83
4.9 Scatter plots of the three regime specific μ and σ from (a) fitting to test data, and (b) random sample from the generative model, using the CTMC model on AUD-USD data. Colours represent three different regimes. 88
4.10 Average scores of the CTMC model when trained on a range of different sized training sets and tested on the period from January 15th, 2019 to December 30, 2019. 89
4.11 Sample of three randomly generated surfaces using the CTMC-VAE for each of the three currency pairs. 90
4.12 Two empirical σ deviations of each latent dimension of a VAE trained on the training set for the EUR-USD currency pair. Each row represents changing the value of one latent dimension while holding all others constant. Each panel represents a cross section of the implied volatility surface taken at a specific time to maturity (1M, 1Y, and 5Y). Within each panel, different coloured curves represent different levels for each latent dimension. 94
4.13 Two empirical σ deviations of each principle component of applying PCA to the encoded latent vectors of historical surfaces using a VAE trained on the training set for the EUR-USD currency pair. Each row represents changing the value of one latent dimension while holding all others constant. Each panel represents a cross section of the implied volatility surface taken at a specific time to maturity (1M, 1Y, and 5Y). Within each panel, different coloured curves represent different levels for each latent dimension. 95

PREVIEW

Chapter 1

Introduction

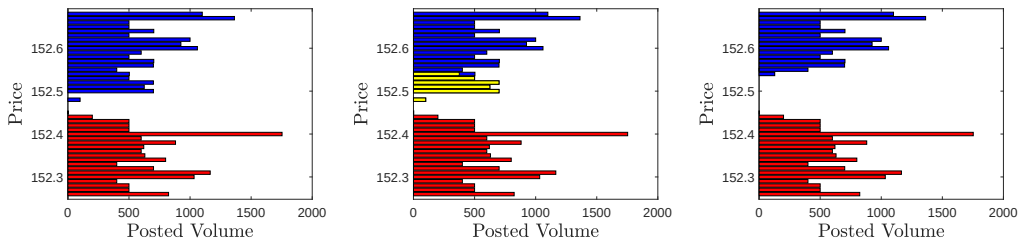
From dusty old ledgers to electronic storage, from over-the-counter bargaining to millisecond trades, financial markets have undergone dramatic changes over the last century. Where once traders needed to physically interchange currency for stock certificates, now such strenuous procedures can be done with click of a button. Electronic stock exchanges have replaced antiquated face-to-face dealings with near-instant transaction servers. This change brought about a revolution to how financial markets are viewed, with a increase in reach, scope, and ease-of-use.

With changes in the market, comes changes in strategies to exploit the market. Technological advances have led to information that once took days, months, if not years before reaching traders, to become public knowledge within hours. This rapid diffusion of information leads traders who can act upon them in the most timely manner to have immense advantages when it comes to trading on the electronic exchanges. Presently, the time between receipt of news (sometimes called *alpha signals*) and the execution of relevant trades are so minuscule that it becomes impossible for humans to physically react. Thus a new class of trading strategies known as *algorithmic trading* (AT) was born. AT refers to the process of executing market trading strategies autonomously through the use of computers. These AT strategies do not rely on human responses but instead real-time data fed directly into the computer. Typically human involvement is limited to initial construction, calibration, and occasional evaluation.

PREVIEW

1.1 The Limit Order Book

Traders may execute two main order types on modern electronic stock exchanges: *limit orders* and *market orders*. A limit order(LO) represents the intention to buy or sell at most a fixed number of shares at a specified price. If there are no shares available at that specified price (or better), then the limit order is posted onto the *limit order book* (LOB) and remains there until the trader either cancels the order or it is filled by another trader. Therefore LOs have a price guarantee, but are not guaranteed to be executed. Traders who post limit orders are said to provide liquidity to the market as they provide a pool of shares at different prices for other traders to either buy or sell. A market order(MO) represents a fixed number of shares to buy or sell, and is matched by the best available LOs remaining in the LOB. The lowest available sell LO is known as the *ask price* and the highest available buy LO is known as the *bid price*. The difference between the ask and bid is known as the



(a) Before MO Arrival (b) Buy MO walking the LOB (c) After MO Arrival

Figure 1.1: The LOB of Facebook Inc. stock at 10:36 on 28 Mar, 2018 as a buy MO of size 3,000 arrives. The blue (red) bars represent available volume of sell (buy) orders at the corresponding price. The yellow bars in the middle panel indicate which LOs match the incoming MOs.

spread and the average of the ask and bid price is known as the *mid-price*. The minimum change in the bid or ask is called the *tick size*. Figure 1.1 illustrate a snapshot of the LOB for INTC as a buy MO for 3,000 shares walks the LOB. Before the MO arrives, the LOB has a spread of 1 tick, the bid and ask are \$48.40 and \$48.41, respectively. After the MO walks the LOB, the bid and ask are \$48.40 and \$48.46, respectively, and the spread widens to 6 ticks.

The share prices in our data are "small" relatively to the tick size, and hence the spread is typically 1 tick – such stocks are called *large tick stocks*. In many situations, one can typically approximate all execution prices by the mid-price and ignore the spread. For most of the analysis done in this thesis, we avoid the cost associated with walking the LOB by enforcing a "transaction cost" penalty on the size of any one order. This mimics the costs associated with "walking the LOB" by imposing a penalty on the amount of shares executed by the trader over any short interval.

1.2 Algorithmic Trading

Algorithmic trading strategies, while all attempt to maximize net profits, can be broadly separated into three distinct categories based on their goals and limitations:

Market Making often refers to strategies that seek to profit by providing liquidity to other market participants while minimizing exposure risk from holding a large positive or negative position of a particular stock. Intuitively, these strategies aim to make a profit by exploiting the differences between the bid price and ask price, effectively profiting off of the spread. They typically provide liquidity by posting limit orders on both sides of the LOB to capitalize on large market orders. As the margins are often very narrow, only large institutional investors participate. We will not be focusing on this facet of algorithmic trading in this thesis.

Optimal Execution are strategies that seek to minimize the costs, and as a result maximize the profits, associated with walking the LOB when either attempting to sell or buy a large number of shares. These strategies are often employed by brokers who are given a specific number of stocks to execute over a set time horizon. The brokers are then tasked with the job of using a combination of LOs and MOs over the given time frame to achieve the desired result, often limited to a single direction (buy vs. sell). We will tackle this specific problem in Chapter 2 of this thesis.

Statistical Arbitrage algorithms attempt to profit by trading on short-term price changes caused by exogenous information or market inefficiencies in the traditional "buy low, sell high"

PREVIEW

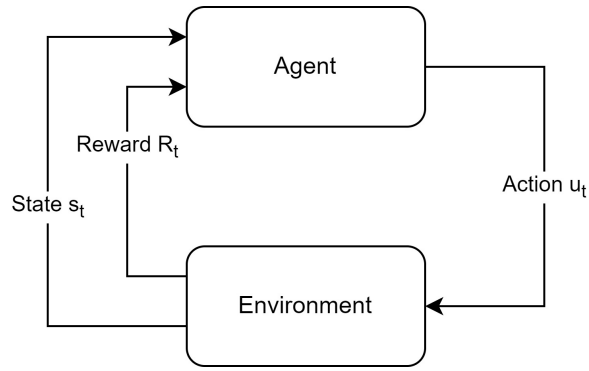


Figure 1.2: Graphical representation of the interactions between the environment and agent.

methodology. Examples of this include alpha signal traders, volatility arbitrage, or cross-market arbitrage. These strategies are often highly data and model reliant as they heavily rely on price forecasts for the traded upon stock. There is no general class of individuals that specialize in statistical arbitrage as they can vary from day-traders to large institutional investors. We study a particular example of this problem in Chapter 3 in the context of a multi-agent stochastic game and develop a new methodology to solve a general class of such games.

1.3 Reinforcement Learning

Reinforcement Learning (RL) is one of machine learning's (ML) three main paradigms, alongside supervised and unsupervised learning. It has had a rich history with numerous key achievements such as Bellmann's work on dynamic programming in 1957 [8] and Q-Learning first introduced by Watkins in 1989 [110]. It can be vaguely classified as learning and optimization done through interactions with the environment. Someone who utilizes RL, known as a RL agent, learns from the consequences of their actions, rather than being explicitly taught what is correct versus incorrect. The agent first observes the environment and then make a decision either randomly (exploration) or by leveraging prior experiences (exploitation) to interact with the environment. The agent then observes the results of their earlier interaction, usually through a reward function, and records this experience to improve future decisions. This interaction loop can be seen graphically in Figure 1.2. Ultimately, the goal of the RL agent is to maximize the total (discounted) rewards obtained across a given time horizon. The transitions the environment undergoes after each interaction is dependent both on the agent's actions and the environment's natural dynamics, however, given an initial state of the system and the actions of the agent, the environment must be conditionally independent of previous states and actions. Such a system is known as a Markov Decision Process (MDP).

Formally, a MDP is a tuple $(\boldsymbol{\mathcal{S}}, \boldsymbol{\mathcal{U}}, \boldsymbol{P}, \boldsymbol{\mathcal{R}})$ over some time horizon $t \in \{0, 1, \dots\}$ where:

- $\boldsymbol{\mathcal{S}}$ is the set of possible states,
- $\boldsymbol{\mathcal{U}}$ is the set of possible actions,
- $\boldsymbol{P}_u(s, s') = P(s_{t+1} = s' | s_t = s, u_t = u)$ is the probability of transition from state s to s' given the agent's action u,

PREVIEW

- $\boldsymbol{\mathcal{R}}(s, u)$ the immediate rewards obtained by the agent from the environment as it transitions from state s through the action u.

The goal of the RL agent is to maximize the total expected rewards given a policy $\pi(u|s)$, defined as a probability distribution over the actions $u \in \boldsymbol{\mathcal{U}}$ given the current state $s \in \boldsymbol{\mathcal{S}}$, some discount factor $\gamma \in (0, 1)$, and initial condition $s_0 = s$:

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \underset{u \sim \pi, s \sim \boldsymbol{P}_u}{\mathbb{E}} \left[\sum_{t=0}^{\infty} \gamma^t \, \boldsymbol{\mathcal{R}}(s_t, u_t) \middle| s_0 = s \right]. \tag{1.1}$$

Define the *value function* $V_\pi(s)$ of a state s under policy π as the total expected return given the environment starts in state s and follows π thereafter:

$$V_\pi(s) = \underset{u \sim \pi, s \sim \boldsymbol{P}_u}{\mathbb{E}} \left[\sum_{t=0}^{\infty} \gamma^t \, \boldsymbol{\mathcal{R}}(s_t, u_t) \middle| s_0 = s \right] \tag{1.2}$$

Under the assumptions of the environment being an MDP, the Bellman Principle [8] applies and thus we can write:

$$V_\pi(s) = \sum_{u \in \boldsymbol{\mathcal{U}}} \pi(u|s) \sum_{s' \in \boldsymbol{\mathcal{S}}} \boldsymbol{P}_u(s, s')(\boldsymbol{\mathcal{R}}(s, u) + \gamma \, V_\pi(s')). \tag{1.3}$$

For optimal deterministic policies $\pi(s)$, which were proven to always exist in a MDP setting in [93], we can use the *Bellman optimality equation* which states that the optimal value function $V_{\pi*}(s)$ is equal to the expected rewards obtained by the agent by taking the most greedy action from each state:

$$V_{\pi*}(s) = \max_{\pi} V_\pi(s) = \max_{u \in \boldsymbol{\mathcal{U}}} \sum_{s' \in \boldsymbol{\mathcal{S}}} \boldsymbol{P}_u(s, s')(\boldsymbol{\mathcal{R}}(s, u) + \gamma \, V_\pi(s')), \quad \forall s \in \boldsymbol{\mathcal{S}}. \tag{1.4}$$

There are numerous methods of solving these equation if the transition dynamics are known, such as value iteration [8], however estimating the transition dynamics directly is usually data inefficient. Thus a variation of the Bellman equation can be used for the Q-value function defined as:

$$Q_\pi(s, u) = \sum_{s' \in \boldsymbol{\mathcal{S}}} \boldsymbol{P}_u(s, s') \left(\boldsymbol{\mathcal{R}}(s, u) + \gamma \sum_{u' \in \boldsymbol{\mathcal{U}}} \pi(u'|s') \, Q_\pi(s', u') \right), \quad \forall s \in \boldsymbol{\mathcal{S}}, u \in \boldsymbol{\mathcal{U}}. \tag{1.5}$$

This leads to the Bellman equation for Q-value functions:

$$Q_{\pi*}(s, u) = \sum_{s' \in \boldsymbol{\mathcal{S}}} \boldsymbol{P}_u(s, s') \left(\boldsymbol{\mathcal{R}}(s, u) + \gamma \max_{u' \in \boldsymbol{\mathcal{U}}} Q_{\pi*}(s', u') \right), \quad \forall s \in \boldsymbol{\mathcal{S}}, u \in \boldsymbol{\mathcal{U}}. \tag{1.6}$$

The Q-value function is easier to estimate as it is a function in two dimension (s and u), instead of three (s, s', u) in the case of transition dynamics, and thus it is used more often in practice.

1.3.1 Q-Learning

In classical Q-Learning we approximate the Q-Value function defined in Equation (1.6) using a discretized grid across both the range of feasible actions and states for the MDP. We then assign a different function value for each state-action pair, stored in a matrix format represent in figure

PREVIEW

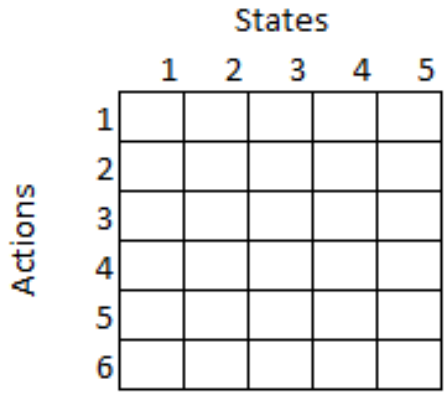


Figure 1.3: Matrix representation of the Q-Value function for a problem with five possible discretized actions and six states.

Figure 1.3. One key disadvantage of approaching problems with continuous action and state spaces using a matrix representation of the Q-value function is the curse of dimensionality. Increasing the dimensionality of the feasible action and state space correspondingly increase the dimensions of the matrix representation of the Q-function. This in turn increases the total number of entries of the matrix exponentially, which quickly becomes untenable. Another downside is in the case of continuous action and state spaces, discretization needs to be performed which may induce biases depending on the procedure. In modern RL, neural network approximations are used instead, including deterministic policy gradient algorithms [99] and value function based approximations similar to the ones used in Chapter 2.

The basis of the Q-Learning algorithm is the trade-off between exploration of the state-action space and exploitation of the best policy for the agent at any given location in the state space. The area of possible exploration strategies has been intensively studied by numerous disciplines across mathematics and the sciences. For a survey of some possible methods, see [75]. The method that is most well known and ubiquitously connected to classical Q-Learning is the ϵ-greedy algorithm.

An exploration strategy is known as ϵ-greedy when at iteration $k \geqslant 1$ the agent either:

- Selection a random action u_k with probability ϵ_k, otherwise
- Select the current best policy $u^*(s) = \text{argmax}_{u \in \boldsymbol{\mathcal{U}}} Q_\pi(s, u)$

The classical Q-Learning algorithm using an ϵ-greedy algorithm can be summarized as follows in Algorithm 1 for some positive sequence $\alpha_k \to 0$ such that $\sum_k \alpha_k = +\infty$ and $\sum_k \alpha_k^2 < +\infty$. This procedure is shown to converge to the optimal policy and its corresponding Q-value function by [76].

1.3.2 Dyna-Q Learning

One of the issues that traditional Q-Learning often faces is data inefficiency. We can see from Algorithm 1 that each update to the Q-function requires one experience, defined as a single interaction of the agent with the environment. Thus to accurately approximate the Q-function over all state-action pairs requires a significant amount of experience which may not necessarily be available for

PREVIEW

```
Initialize: Q(s, u)
for Iteration k ← 1 to K do
    Initialize state s
    for Iteration t ← 1 to T do
        Select ϵ-greedy action u
        Take action u, observe s′ and reward r
        Update Q-value function according to:
            Q(s, u) ← (1 − α_k) Q(s, u) + α_k [r + γ max_{u′∈𝒰} Q(s′, u′)]
        Update s ← s′
    end
end
Return : Q
```

Algorithm 1: Classical Q-Learning Algorithm

non-simulated environments. Sutton proposes a hybrid model based approach, known as Dyna-Q Learning, that takes advantage of the fact that state transitions are often much easier to approximate than the Q-function [102].

Dyna-Q Learning leverages a model which approximates the environment by constructing a function which, when given a state-action pair, outputs the resultant state and the expectant rewards obtained by the agent. This model is updated whenever the agent interacts with the environment. In contrast, the Q-function is updated only using samples from this model instead of directly from experiences. This allows the algorithm to perform significantly more update iterations to the Q-function without additional experiences. This is especially important in environments which involve considerable costs for the agent to interact with, such as financial trading, or engineering. The algorithm is described in Algorithm 2.

```
Initialize: Q(s, u)
for Iteration k ← 1 to K do
    Initialize state s
    for Iteration t ← 1 to T do
        Select ϵ-greedy action u
        Take action u, observe s′ and reward r
        Update Q-value function according to:
            Q(s, u) ← (1 − α_k) Q(s, u) + α_k [r + γ max_{u′∈𝒰} Q(s′, u′)]
        Update Model using transition (s, u) → (s′, r)
        for sample n ← 1 to N do
            Sample s̃ from previously visited
            Sample ũ from previous actions at state s̃
            Sample (s̃, ũ) → (s̃′, r̃) using model
            Update Q according to:
                Q(s̃, ũ) ← (1 − α_k) Q(s̃, ũ) + α_k [r̃ + γ max_{u′∈𝒰} Q(s̃′, u′)]
        end
        Update s ← s′
    end
end
Return : Q
```

Algorithm 2: Dyna-Q Learning Algorithm

PREVIEW

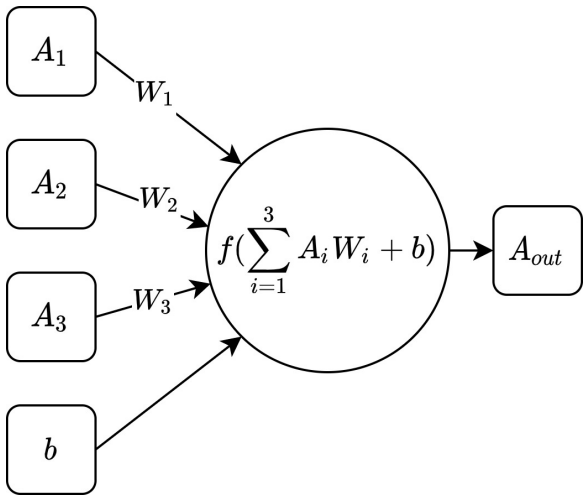


Figure 1.4: Graphical representation of a single neuron inside of neural network. The arbitrary non-linear function f is applied to a linear combination of upstream nodes plus a bias and outputs its value as a downstream node.

1.4 Neural Networks

A neural network is typically defined as a network of biological or artificial neurons, sometimes called nodes. An artificial neural networks usually consists of multiple layers of neurons (known as the depth) each having multiple nodes (known as the width). Each neuron is represented as some non-linear function f (known as the activation function) acting upon some linear combination of the output of upstream neurons. Some examples of commonly utilized f include sigmoid, ReLU [2], and SiLU [36]. We can see in Figure 1.4 an example of a single node with three input upstream nodes.

In a fully connected neural network (FC), all neurons in adjacent layers are connected to each other however, no neurons within each layer is connected to each other. There are numerous works on the structure of neural networks, each with their own particularities, however the FC network still remains as one of the most ubiquitous due to its simplicity.

It has been shown using the Universal Approximation Theorem [53] that neural networks with arbitrary widths and bounded depths with sigmoid activation functions can approximate any continuous function arbitrarily closely. Recently, the variation for networks with bounded width and arbitrary depth was proven by Zhou Lu et al. in 2017 [74]. Neural networks are now a cornerstone of modern machine learning and, together with back-propagation [41] and stochastic gradient descent, serves as the foundation to a myriad of algorithms and methods by approximating complex analytical functions.

In general using neural networks can be divided into two phases: training and inference. During training, the user defines a loss function, usually quantifying the accuracy of the neural network along with encouraging desired behaviours, which is minimized via stochastic gradient descent or its variations. The gradients used are usually determined through the use of automatic differentiation [83] which takes advantage of the fact that all computational processes, no matter its complexity, is executed using a sequence of basic arithmetic operations (i.e. $+$, -, $\times$, $\div$) and continuous elementary functions (i.e. *sin*, *cos*, *exp*, etc) which have simple derivatives. By representing these operations in a directed acyclic graph, the chain rule can be used to calculate ("back-prop") the gradients of

PREVIEW

arbitrarily complex functions.

During inference, a neural network freezes the weights and biases that minimized the loss function during training and uses given input to evaluate its corresponding output. This is done by computing the weights and biases applied to each input, applying the corresponding activation functions at each neuron, and then relying the result forwards to the next layer. This is done iteratively through all layers of the network and eventually arrive at the output from the final layer of the neural network which is denoted as the result.

1.5 Literature Review

Algorithmic trading is a well-studied problem with numerous classical and modern approaches. Traditionally, traders engaging in both optimal execution and statistical arbitrage propose stochastic models (i.e. Ornstein-Uhlenbeck processes, etc) based on empirical observations of historical data to estimate model parameters, define some performance criteria they wish to optimize, then solve the problems analytically using methods in stochastic control theory. One of the earliest examples is the Almgren-Chriss [3] approach and its modern generations found in [24]. However these approaches become intractable in complex pricing models without analytical solutions.

Modern ML approaches to the single agent case mainly revolves around applying RL algorithms in an appropriate manner. One of the most popular methods, Q-learning, has been applied to the optimal execution problem in both [85] and more recently in [49]. However these methods are often restricted by the pointwise nature of Q-learning. Our work in Chapter 2 is one of the first to adapt and modify modern Deep Q-Learning [79] [47] in a optimal execution framework. Other recent approaches include viewing paths of stochastic processes as "signatures" which are then optimized to obtain the optimal strategy [25].

Algorithmic trading in the multi-agent setting, or more specifically the study such equilibria in systems with interacting agents has a storied past throughout the natural and social sciences. Traditionally traders take a mean-field approach to solving such problems [28] [60]. Finite player solutions often involve solving systems of FBSDEs (Forward-Backward Stochastic Differential Equations) under certain modeling assumptions [84]. RL usage in multi-agent algorithmic trading is not often studied, however in traditional RL literature, methods have been developed for multi-agent games but are often restricted to the case of zero-sum games, which is often not the case in algorithmic trading. For a full survey of such methods see [16].

Recently extensions of RL to general sum games with methods such as fictitious play [56] and other fixed-point methods [66] have been developed. Another approach, introduced by Hu and Wellman [55], uses Q-learning as a basis for obtaining Nash equilibria in general-sum stochastic games by repeatedly computing a local Nash equilibrium over states and show its convergence for finite game and action spaces. However, this approach becomes computationally infeasible for most modern problems. We develop a computationally efficient method in Chapter 3 to compute the Nash equilibria for general-sum stochastic games.

Algorithmic hedging using machine learning methods is an under-explored field in finance. There has been some recent attempts at "deep derivative hedging" such as [17] and [19], however, their models are largely reliant on simple stochastic volatility models (i.e. Heston, Brownian, etc) for derivative pricing. Such models often place significant restrictions on shape of the volatility surfaces

PREVIEW

of derivatives which often do not reflect the implied volatility (IV) surfaces found in historical data. We first tackle the problem of simulating historically accurate volatility surfaces in Chapter 4. The volatility surface generator can then replace the stochastic volatility models used in [19] to develop a more flexible algorithm.

Traditionally modelling IV surfaces are often driven by setting up stochastic differential equations (SDEs) which attempts to reflect historical dynamics while remaining arbitrage-free. These include local volatility models [35], stochastic volatility models [51, 46], stochastic local volatility models [97], jump-diffusion models [33], and regime switching models [18], among many others. These approaches typically enforce risk-neutrality by design but are often constrained by numerous underlining assumptions. However, recent alternatives make use of non-parametric models to approximate the IV surfaces directly without making assumptions on the underlying dynamics, examples include: support vector machines (SVMs) [112], variational autoencoders (VAEs) [10], and Gaussian processes (GPs) [30]. These methods either assume the generated surfaces do not induce significant enough arbitrage for traders to execute on, or enforces certain no-arbitrage conditions to the generation or training procedure. We take a more hybrid approach in Chapter 4 where we first use classical stochastic volatility models to estimate sets of parameters which generate arbitrage-free surfaces then apply generative models to the space of model parameters to generate samples.

1.6 Outline and Contributions

The outline of this thesis is summarized as follows. In Chapter 2 we defined the single-agent optimal execution problem. We then introduce a modified Double DQN approach to estimate the optimal actions of a trader using a wide range of LOB features, other trading signals, and available execution actions. We use a fully connected Neural Network trained using Experience Replay to approximate the Q-value function of the agent. We apply our model to several different stocks and evaluate its performance when compared with a baseline approach.

This chapter, based on [86], is one of the first to adapt and modify the framework of Deep Q-learning to the optimal execution problem by estimating the Q-function used in Q-learning with a deep neural network which allows us to interpolate between actions and states as well as reduce the storage costs associated with classical Q-learning.

Chapter 3 develops a generic approach to solving for Nash equilibria in multi-agent games and applies it to the case when multiple agents impact the market and are aiming to perform statistical arbitrage. We develop a general Q-learning based approach using local linear-quadratic expansion of the Q-value function motivated by the $iLQG$ framework of [106, 43] (which deals with the single agent setting only) and the Nash Q-learning algorithm of [55] (which is tractable in only very small finite state/action settings). We also provide several simplifications that arise in label-invariant games. We then present a complex statistical arbitrage problem with no known analytical solution and compare our algorithm's performance to a benchmark method using fictitious play. Finally, we consider several variations of the original statistical arbitrage problem and explore the Nash-optimal policy for all agents in each case.

The majority of the existing literature on RL is dedicated to single-agent games. Most modern approaches follow either a deep Q-learning approach, policy gradient methods, or some mixture thereof. RL methods have also been developed for multi-agent games, but are for the most part

PREVIEW

restricted to the case of zero-sum games. This chapter presents a unique extension of the Deep Q-Learning framework to general-sum games that is capable of efficiently learning Nash equilibria in complex and practically relevant settings.

Chapter 4 takes the first steps to constructing an algorithmic hedging algorithm by developing a method for generating arbitrage-free implied volatility (IV) surfaces which are consistent with historical data. The generated IV surfaces can then be used to derive derivative prices which could then be used as the basis for a reinforcement learning algorithm. We propose a hybrid method that uses classes of SDE models that are by definition arbitrage-free yet flexible enough to fit arbitrary IV surfaces. We focus on two classes of SDE models: regime switching models and Lévy additive processes. By projecting historical surfaces onto the space of SDE model parameters, we obtain a distribution on the parameter subspace faithful to the data on which we then train a VAE. Arbitrage-free IV surfaces are then generated by sampling from the posterior distribution on the latent space, decoding to obtain SDE model parameters, and finally mapping those parameters to IV surfaces. We further refine the VAE model by including conditional features and demonstrate its superior generative out-of-sample performance on real-world foreign exchange data for several currency pairs.

Typically classical methods of IV surface generation enforce risk-neutrality by design but are often constrained by numerous underlining assumptions while more modern approaches either assume the generated surfaces do not induce significant enough arbitrage for traders to execute on, or enforces certain no-arbitrage conditions to the generation or training procedure. This chapter presents a hybrid approach of generating inherently arbitrage-free IV surfaces with minimal model assumptions that is capable of accurately capturing key features of historical data for use in data augmentation or simulation.

Chapter 5 summarizes the contributions of this thesis and future work to be done to incorporate the IV surface generator to solve an algorithmic hedging problem. We also propose other branches of possible research following the work of this thesis.

PREVIEW

1.7 Remarks

As a note to the reader, the mathematical notation of similar concepts across chapters may vary. As a result of this, each chapter is designed to be self contained and does not rely on any preceding chapter's notation nor concepts. Thus some repetition, especially in the definition of the Q-value function and other RL concepts, can be found in some of the chapters. Certain RL concepts may be repeated within some of the following chapter, however a more comprehensive overview can be found in Section 1.3.

The majority of this thesis does not involve any complex proofs and thus will be presented directly inside the body of each chapter. Detailed tables and figures of results can be found at the end of each chapter with appropriate references from their respective sections.

The majority of this thesis is composed of content from a number of papers produced over the course of 2018-2021. Chapter 2 is based on [86], currently published under *Applied Mathematical Finance*. Chapter 4 is based on [87], currently under review with *SIAM Journal on Financial Mathematics*. Chapter 3 is the only chapter not yet contained within any standalone paper, though we are planning on changing this soon.

Chapter 2

Double Deep Q-Learning for Optimal Execution

2.1 Introduction

Financial markets are highly complex stochastic systems with significant heteroskedasticity. The problem of how to optimally execute large positions over a given trading horizon is an important problem faced by institutional investors, banks, and hedge funds. Naïvely rebalancing a portfolio could result in significant adverse price movements as other intelligent traders may read off the signal. Investors must balance trading quickly and obtaining poor execution prices, with trading slowly which exposes them to unknown market fluctuations.

Agents are often exposed to a plethora of information including prior stock prices and other market conditions. Determining the best trading policy in the presence of all of this information is an important part of the algorithmic trading literature. Traditionally, researchers propose a stochastic model based on empirical observations, such as an Ornstein-Uhlenbeck or stochastic volatility process, use historical data to estimate model parameters, such as the volatility, mean-reversion level and rate, propose a performance criteria which they aim to maximize, and then solve the problem analytically using methods in stochastic optimal control. One of the earliest works in this vein is the Almgren-Chriss [3] approach, where the authors assume prices are Brownian motion. There has been many generalizations including of this approach to account for a variety of market features, see, e.g., [44], [23], and [29], and the graduate textbook [24] for a modern treatment.

However, in the case of more complex price models without analytical solutions, or even non-parametric models, a different approach is necessary. Reinforcement learning [105] attempts to learn optimal policies for sequential decision problems by optimizing a cumulative future reward function with few modeling assumptions (such as Markovian structure of the state space). The most popular reinforcement learning algorithm is Q-learning [109] which has been applied to the optimal execution problem in [85] and [49].

One of the major drawbacks to the Q-learning algorithm is that it estimates the optimal policy on a pointwise basis. This severely limits the ability to interpolate between possible actions, and therefore does not allow for effective generalization. Moreover, the algorithm's storage space scales exponentially with respect to the dimension of the state space. To address this issue, we modify

PREVIEW

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.